

PRD: Mini Credit Application

Engineering Case Study

1. Overview

This document describes a take-home engineering case study. The goal is to build a mini credit application platform where a vendor can (a) create or upload a credit application, (b) send it to a recipient for completion, and (c) review and decision the submitted application. The exercise is designed to evaluate product thinking, system design, pragmatic trade-offs, and the ability to ship a working end-to-end product in a tight timebox.

AI-assisted coding is expected and encouraged. Use whatever tools, frameworks, and stack you are most productive with.

2. Timebox & Deliverables

Timebox: 10–15 hours total. If you hit 15 hours, stop where you are and submit what you have. Do not exceed the cap — we care more about how you use your time than about feature completeness.

Deliverables:

- A GitHub repository link (public or invite us) OR a ZIP file of the source code.
- Working prototype deployed in production
- A loom video of the demo
- **README.md** including: (a) setup/run instructions, (b) assumptions and decisions you made, (c) known gaps or shortcuts, and (d) a “Next 10 Hours” section describing what you would build or refine next if you had another ten hours.
- A short note on how you used AI in the build (tools, workflow, what you accepted/rejected).

Follow-up interview: A 45-minute live session where we walk through the code together, ask questions, and run a live schema design exercise together.

3. Users & Core Roles

- **Vendor** — the party offering credit terms. Creates the application template, sends it out, and reviews submissions. No authentication required.
- **Customer (Recipient)** — the business applying for credit. Receives a link via email, fills out the form, and submits.

4. Functional Requirements

4.1 Vendor landing

The vendor lands on a simple home screen (no auth) with two primary actions:

1. Create / set up a credit application (build new or upload existing).
2. Send a completed application to a recipient.

A basic applications dashboard (see 4.6) should also be reachable from this screen.

4.2 Building the credit application

The vendor can build a credit application with the following fields:

- Name of company (text)
- DBA — doing business as (text)
- Country of incorporation (text or dropdown)
- Website (URL)
- Two trade references, each with: business name, engagement start date, engagement end date, contact name, contact email, contact position
- Credit amount requested (dollar amount, text)
- Credit term limit requested (dropdown: Net 10 / Net 20 / Net 30)
- Billing contact: name, email
- Size of the business by revenue (dropdown: 1–10M / 10–100M / 100–250M / 250–500M / etc.)

4.3 Uploading an existing application

As an alternative to building from scratch, the vendor can upload an existing credit application (e.g. a PDF) and the system should parse it using AI into an editable form.

- A sample upload can be used as reference [here](#)
- If the uploaded file contains fields the system does not support, ignore them but clearly notify the vendor which fields were dropped.

4.4 Sending the application

- The vendor enters the recipient's name and email and sends the application.
- The recipient receives an email with a unique link to fill out the form.
- The link should be scoped to a single application instance.

4.5 Recipient submission

- Opening the link renders the credit application form.
- On submit, the system records the submission and notifies the vendor by email with a link to the submitted response.

4.6 Vendor review & decisioning

The vendor can open a submitted application and take one of three actions:

- Approve as submitted.
- Approve with adjusted term limits (amount and/or Net term).
- Reject.

The outcome and any adjustments should be persisted on the record.

4.7 Applications dashboard

A basic table/CRUD view listing all applications and their current status. Statuses should include at minimum:

- Draft
- Sent
- Submitted
- Approved
- Approved with adjustments
- Rejected

5. Extra Credit (Optional)

These are nice-to-haves. Do not attempt them at the cost of core flow quality.

5.1 Apollo prefill

When the customer opens the application link, prompt them first for their business name. Use the Apollo.io API to look up the business and prefill relevant fields (country, website, revenue band, etc.) so the customer only has to correct/confirm.

- **API:** apollo.io
- **API key:** aORFIhKQFDfdiRcmPNjQYA

5.2 AI decision summary

On vendor review, generate an AI-authored recommendation summary that suggests approval, rejection, or a term-limit adjustment based on the information provided in the submission (trade references, revenue band, requested amount, etc.). The vendor remains the final decision-maker; the AI output is advisory.

6. Out of Scope

- Authentication, user management, or multi-tenant isolation.
- Production-grade email deliverability (a dev sandbox like Resend/Mailtrap/Postmark sandbox is fine; console logging is acceptable if the link is surfaced).
- Production-grade security, rate limiting, or compliance (SOC2, PCI, KYC, etc.).
- Pixel-perfect design — clean, usable UI is enough.

- Any integration beyond Apollo (for extra credit) and a single LLM provider of your choice (for extra credit).

7. Tech & Implementation Guidance

- **Stack:** Use whatever you are fastest in. Any language, any framework, any datastore.
- **AI-assisted development is expected.** Use Cursor, Claude Code, Copilot, Windsurf, etc. We care about the final product and your judgment, not whether every line was hand-written.
- **Data model:** Be prepared to defend your schema in the interview — we will very likely extend it live together.
- **Deploy:** Optional. A local run via a single command is sufficient.
- **Email:** Any dev-friendly service, or log the link to the console if time-constrained — just make sure the flow works end-to-end.

8. Evaluation Criteria

We will weigh the following, roughly in order:

3. End-to-end flow works: vendor builds → sends → customer submits → vendor decisions.
4. Product judgment: what did you choose to build, cut, or defer inside the timebox, and why.
5. Code clarity and structure: someone else could read this and extend it.
6. Data model: entities, relationships, statuses, and how extensible it is.
7. Quality of the README, assumptions, and the “Next 10 Hours” plan.
8. Thoughtful use of AI in the build.
9. Extra credit features (if attempted without sacrificing the core).

9. Follow-up Interview (45 min)

A live session covering:

- Walkthrough of the codebase and key decisions (~15 min).
- Questions on trade-offs, AI usage, and what you cut (~10 min).
- Live schema / design extension exercise — we will pose a new requirement and evolve your data model and API together on the fly (~20 min).

10. Submission

- GitHub repo link or ZIP file of source.
- Deployed production code and a loom video of the demo

- README.md with setup, assumptions, known gaps, AI-usage notes, and the “Next 10 Hours” section.
- Any sample uploads, seed data, or credentials the reviewer needs to run the app locally.

11. Requirements Summary

Area	Requirement
Auth	None.
Vendor home	Two primary actions: create application, send application. Dashboard accessible.
Create app	Full field set (see 4.2) OR upload existing form and auto-extract fields.
Send	Email recipient with a unique, scoped link.
Recipient	Fill & submit form. Vendor notified by email on submit.
Review	Approve / approve with adjusted terms / reject.
Dashboard	CRUD table with statuses: draft, sent, submitted, approved, approved w/ adjustments, rejected.
Extra credit 1	Apollo.io prefill by business name.
Extra credit 2	AI-generated decision recommendation summary.
Timebox	10–15 hours. Hard stop at 15.
Deliverables	Repo or ZIP + README (assumptions, gaps, next-10-hours) + AI-usage notes.
Interview	45 min: walkthrough, Q&A, live schema exercise.